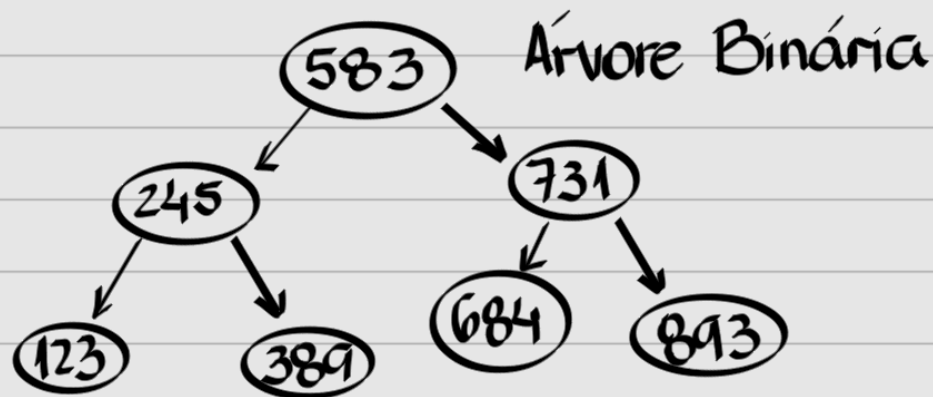


Árvores

- Organização e busca de dados
- Gerencia de arquivos
- Composta por nós, onde cada um possui dois ponteiros
- Raíz: nó inicial, mais alto
- Folhas: últimos nós, que não possuem folhas (NULL)



Inserção:

```
struct Node {  
    int valor;  
    Node *left, *right;  
}Node;
```

Se um elemento é menor, por ser árvore binária, entra no ponteiro left. Se não, right.

```
Node* insert1(Node *raiz, int num) {  
    if (raiz == NULL) {  
        Node *aux = malloc(sizeof(Node));  
        aux->valor = num;  
        aux->left = NULL;  
        aux->right = NULL;  
        return aux;  
    } else {  
        if (num < raiz->valor)  
            raiz->esquerda = insert1(raiz->esquerda, num);  
        else raiz->direita = insert1(raiz->direita, num);  
        return raiz; //garante que não altera a raíz original
```

```
}  
}
```

Print pré-ordem:

```
void print1(Node *raiz) {  
    if (raiz) {  
        printf("%d", raiz->valor);  
        print1(raiz->esquerda);  
        print1(raiz->direita);  
    }  
}
```

Print ordenado:

```
void print2(Node *raiz) {  
    if (raiz) {  
        print2(raiz->esquerda);  
        printf("%d", raiz->valor);  
        print2(raiz->direita);  
    }  
}
```

Inserção sem return:

```
void insert2(Node **raiz, int num) {  
    if (*raiz == NULL) {  
        *raiz = malloc(sizeof(Node));  
        (*raiz)->valor = num;  
        (*raiz)->esquerda = NULL;  
        (*raiz)->direita = NULL;  
    } else {  
        if (num < (*raiz)->valor)
```

```

    insert2(&((*raiz)->esquerda), num);
    else insert2(&((*raiz)->direita), num);
}
return;
}

```

endereço de um ponteiro

Inserção iterativa:

```

void insert3(Node **raiz, int num) {
    Node *aux = *raiz;
    while(aux) {
        if (num < aux->valor) raiz = &aux->esquerda;
        else raiz = &aux->direita;
        aux = *raiz;
    }
    aux = malloc(sizeof(Node));
    aux->valor = num;
    aux->esquerda = NULL;
    aux->direita = NULL;
    *raiz = aux;
    return;
}

```

avança a iteração

alteramos a var. local raiz

Busca recursiva:

```

Node* busca1(Node *raiz, int num) {
    if (raiz) {
        if (num == raiz->valor) return raiz;
        else if (num < raiz->valor)
            return busca1(raiz->esquerda, num);
        else return busca1(raiz->direita, num);
    } else return NULL;
}

```

returns servem para desempilhar recursão

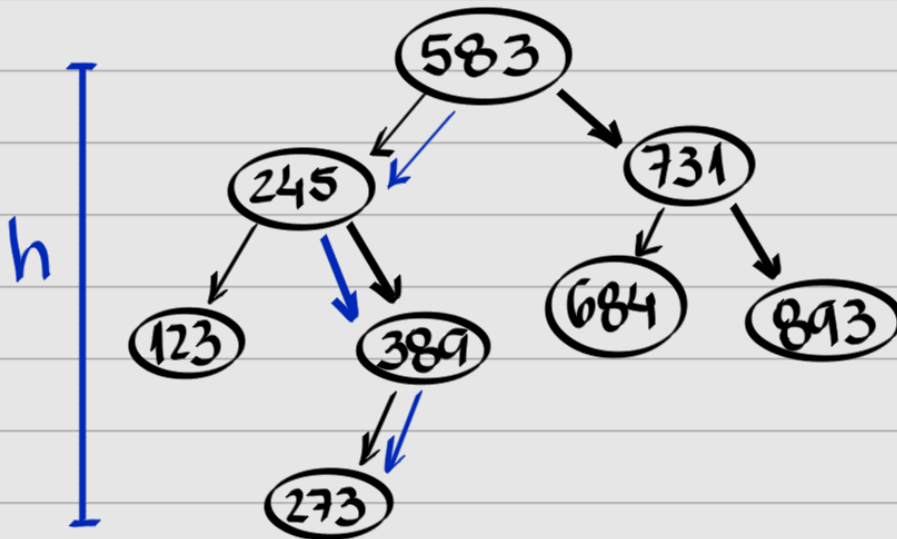
}

Busca iterativa:

```
Node* busca2(Node *raiz, int num) {  
    while (raiz) {  
        if (num < raiz->valor) raiz = raiz->esquerda;  
        else if (num > raiz->valor) raiz = raiz->direita;  
        else return raiz;  
    }  
    return NULL; //caso árvore vazia ou elem. não encontrado  
}
```

var. local da função

Cálculo da altura de uma árvore binária:



- Maior distância da raiz até uma folha

```
int altura(Node *raiz) {  
    if (raiz == NULL) return -1; //trata os casos de apenas 1 nó  
    else {  
        int esq = altura(raiz->esquerda);  
        int dir = altura(raiz->direita);  
        if (esq > dir) return esq + 1;  
        else return dir + 1;  
    }  
}
```

A soma é o incremento dos contadores

```
} //se a árvore for nula, o retorno é -1
```

Quantidade de nós de uma árvore binária:

```
int quant(Node *raiz) {  
    if (raiz == NULL) return 0;  
    else return 1 + quant(raiz->esquerda) + quant(raiz->direita);  
    //return (raiz == NULL) ? 0 : 1 + quant(raiz->esquerda) +  
    //quant(raiz->direita); //com operador ternário  
}
```

Quantidade de folhas de uma árvore binária:

```
int folhas(Node *raiz) {  
    if (raiz == NULL) return 0;  
    else if (raiz->esquerda == NULL && raiz->direita == NULL)  
        return 1;  
    else return folhas(raiz->esquerda) + folhas(raiz->direita);  
}
```

Remoção:

- A remoção pode acontecer em uma folha, em um nó com um filho ou em um nó com dois filhos.

- Código comum:

```
Node* remove(Node *raiz, int num) {  
    if (raiz == NULL) return NULL;  
    else {  
        if (raiz->valor == num) {  
            REMOÇÃO  
        } else {  
            if (num < raiz->valor)
```

```
        raiz->esquerda = remove(raiz->esquerda, num);
    else raiz->direita = remove(raiz->direita, num);
    return raiz;
}
}
}
```

- Remoção de uma folha:

```
if (raiz->valor == num) { //REMOÇÃO
    if (raiz->esquerda == NULL && raiz->direita == NULL) {
        free(raiz);
        return NULL;
    }
}
```

- Para os demais:

```
if (raiz->valor == num) { //REMOÇÃO
    if (raiz->esquerda == NULL && raiz->direita == NULL) {
        free(raiz);
        return NULL;
    } else {
        if (raiz->esquerda != NULL && raiz->direita != NULL) {
            //REMOÇÃO DE NÓS COM DOIS FILHOS
        } else {
            //REMOÇÃO DE NÓS COM UM FILHO
        }
    }
}
```

- Para um filho:

```
else {
    if (raiz->esquerda != NULL && raiz->direita != NULL) {
```

//REMOÇÃO DE NÓS COM DOIS FILHOS

```
} else {  
    Node *aux;  
    if (raiz->esquerda != NULL) aux = raiz->esquerda;  
    else aux = raiz->direita;  
    free(raiz);  
    return aux;  
}  
}
```

- Para dois filhos:

```
else {  
    if (raiz->esquerda != NULL && raiz->direita != NULL) {  
        //escolhemos o nó mais a direita, partindo do filho na  
        //esquerda ou o mais a esquerda partindo do da direita.  
        //apenas trocamos os elementos, removemos um nó  
        //com, no máximo, 1 filho.  
        Node *aux = raiz->esquerda;  
        while (aux->direita != NULL) aux = aux->direita;  
        raiz->valor = aux->valor;  
        aux->valor = num;  
        raiz->esquerda = remove(raiz->esquerda, num);  
        return raiz;  
    }  
}
```

↳ Procura apenas na subárvore à esquerda

RECOMENDO A EXECUÇÃO DE CADA CÓDIGO NO PAPEL